Collection Data Types

- **List**

  ordered, mutable, allow duplicates

- **Tuple**

  ordered, immutable, allow duplicates

- **Set**

  unordered, unindexed, immutable, allow adding items, no duplicate

- **Dictionary**

  unordered, mutable and indexed, no duplicates

Create a List

```
(possible to have different data types as items)
```

- by list built-in function
  list()
- by square brackets
  []

Length of a List

```
len(list)
```

Accessing List Items by Index

```
index[0] to index[-1]
```

Unpacking List Items

```
first_item, second_item, ..., last_item = list
    OR
first_item, *rest list
    ## * store all the rest
```

Slicing Items From a List

```
list[start:end:step]
    #returns a new list
```

Modifying List

| Checking Item in a List

```
"banana" in fruits
    # returns True / False
```

| Update Item

```
fruits[0] = "apple"
    #change item in index 0 to apple
```

| Adding Items

```
- list.append(item) -> add item to the end
- list.insert(index, item) -> add item to the index, other item shifted to
the right
```

| Removing Items

```
- list.remove(item)
- list.pop(index) -> default is the last index
- del list(index) -> remove item in index or index range
- del list -> remove the list
```

| Clearing a List

```
clear() method
```

| Copying a List

```
copy() method
```

| Joining Lists

```
- + operator
    list3 = list1 + list2
```

```
- extend() method
    list.extend(list2)
        #[list1, list2]
```

| Count Items in a List

```
count(item) method
```

| Find Index of an Index

```
index(item) method
```

| Reversing a List

```
reverse() method
```

| Sorting List Items

```
- list.sort(reverse=True/False optional) method
- sorted(list, reverse=True/False) -> create a new list
- sorted(list, key=lambda x: language.count(), reverse=True/False)
```